

Shefa Issachar – Data Analysis (01/06/25 – 30/11/25)

-- Table of Contents:

--- Part 1. Data Presentation P. 4

- 1.1A: Location accuracy distribution (bucketed)
- 1.1B: Payment method distribution (Yavne Branch)
- 1.1C: Average Transaction Value by Hour (Yavne Branch)
- 1.1D: Average transaction value vs. transaction volume by hour (Yavne Branch)
- 1.1E: Weekly revenue vs. transactions (Yavne Branch)
- 1.1F: Average transaction value by payment method (Yavne Branch)

--- Part 2. Metrics P. 7

- 2.1.1: Customer device universe (geolocation)
- 2.1.2 – Payers universe (log_sales; customer_id IS NOT NULL)
- 2.1.3 – No-phone activity (log_sales where customer_id IS NULL)
- 2.1.4: In-store presence features per device
- 2.1.5: Final customer-type classification
- 2.1.6: QA summary counts by customer_type (final validation)
- 2.2.1: Employee device universe (geolocation)
- 2.2.2: Employee role × area exposure (evidence layer)
- 2.2.3: Dominant work area per employee
- 2.2.4: Seniority / consistency signals per employee
- 2.2.5: Role inference (dominant area + seniority rule)
- 2.2.6: Final employee role mapping (one row per employee)
- 2.2.7 (BONUS): Infer employee/customer segment WITHOUT geolocation.role
- 2.3.1: Accuracy outliers (accuracy_m > 30m)
- 2.3.2: App performance (PARKING reception vs 40% benchmark)
- 2.3.3 (BONUS): Additional outliers beyond accuracy_m
- 2.4.1: Identify supplier (delivery) devices (role = delivery_guy)
- 2.4.2 (BONUS): Infer supplier (delivery) devices WITHOUT using geolocation.role

- 2.4.3: Scheduled delivery dates with NO supplier arrival
- 2.5.1: Top 5 customers by visit frequency
- 2.5.2: Top 5 customers by cumulative spend
- 2.6.1: Customer presence by hour × weekday (simple aggregate)
- 2.6.2: Customer presence by hour × weekday (filtered for “real operating hours”)

--- Part 3. Business Recommendations P. 33

- 3A.2.1: Paying-customer load by weekday × hour (log_sales)
- 3A.3.1: Cashiers needed per weekday × hour (from log_sales)
- 3A.3.2: Checkout load by weekday × hour (log_sales)
- 3A.3.3: Peak-hour flag + self-checkout recommendation (decision layer)
- 3A.4.1: Dates with NO supermarket activity (full closure)
- 3A.4.2: Unusually high customer-activity dates (daily spikes)

--- Part 4. Regression Models P. 39

- 4.1: Weekly unique customer devices (Regression 1 dataset)
- 4.2: Basket total vs dwell time (dataset for linear regression)

--- Part 5 – Looker Studio Dashboard P. 40

- 5.1: Heatmap source: customer activity by weekday × hour (customers only)
- 5.2: Daily staffing by shift and role (distinct employee devices)
- 5.3: Weekly unique customer devices (repeat + one_time customers) for trend chart
- 5.4: Raw customer events (so Looker can COUNT DISTINCT device_id within the selected date range)

--- APPENDIX 1 – Supporting Queries P. 44

- Appendix 1.1 – Row count snapshot (all raw tables)
- Appendix 1.2 – Time coverage snapshot (min/max timestamps)
- Appendix 1.3 – Geolocation spatial bounds (lat/lon)
- Appendix 1.4 – Schema snapshot (field names & data types)
- Appendix 1.5 – Null profile for key fields (geolocation + log_sales)
- Appendix 1.6 – Location accuracy bounds (0–100m window)
- Appendix 1.7 – Location accuracy percentiles (0–100m window)
- Appendix 1.8 – Outlier share by 30m threshold (0–100m window)

- Appendix 1.9 - Identifier sanity checks (distinct counts)
- Appendix 1.10 - Duplicate sale_id check (log_sales)
- Appendix 1.11 - Duplicate device_id + timestamp check (geolocation)
- Appendix 1.12 - Duplicate supermarket_id check (supermarkets_il)
- Appendix 1.13 - Duplicate city_id check (Lamas)
- Appendix 1.14 - Clean-table parity check (geolocation_base_clean vs raw)

-- Part 1. Data Presentation (Descriptive Statistics & Key Visuals)

-- 1.1A: Location accuracy distribution (bucketed)

-- **Purpose:** Summarise location accuracy by bucketing accuracy_m into meter bands for a data-quality chart and outlier thresholding.

-- **Method:** Bucket geolocation accuracy_m values, count pings per band, and calculate distribution shares.

-- **Result:** Accuracy outliers: 415,580 / 1,308,624 pings (31.76%). Concentrated in PARKING (54.79%, p99≈114m) and security (68.65%, p99≈149m).

-- **Interpretation:** Precision is uneven by area/device; treat these pings as reliable for presence, but low-confidence for fine movement/path analysis.

```
WITH bucketed AS (
  SELECT
    CASE
      WHEN accuracy_m BETWEEN 0 AND 10 THEN '00-10 m'
      WHEN accuracy_m BETWEEN 11 AND 20 THEN '11-20 m'
      WHEN accuracy_m BETWEEN 21 AND 30 THEN '21-30 m'
      WHEN accuracy_m BETWEEN 31 AND 40 THEN '31-40 m'
      WHEN accuracy_m BETWEEN 41 AND 50 THEN '41-50 m'
      WHEN accuracy_m BETWEEN 51 AND 60 THEN '51-60 m'
      WHEN accuracy_m BETWEEN 61 AND 70 THEN '61-70 m'
      WHEN accuracy_m BETWEEN 71 AND 80 THEN '71-80 m'
      WHEN accuracy_m BETWEEN 81 AND 90 THEN '81-90 m'
      WHEN accuracy_m BETWEEN 91 AND 100 THEN '91-100 m'
      ELSE '100+ m'
    END AS accuracy_bucket,
    CASE
      WHEN accuracy_m BETWEEN 0 AND 10 THEN 1
      WHEN accuracy_m BETWEEN 11 AND 20 THEN 2
      WHEN accuracy_m BETWEEN 21 AND 30 THEN 3
      WHEN accuracy_m BETWEEN 31 AND 40 THEN 4
      WHEN accuracy_m BETWEEN 41 AND 50 THEN 5
      WHEN accuracy_m BETWEEN 51 AND 60 THEN 6
      WHEN accuracy_m BETWEEN 61 AND 70 THEN 7
      WHEN accuracy_m BETWEEN 71 AND 80 THEN 8
      WHEN accuracy_m BETWEEN 81 AND 90 THEN 9
      WHEN accuracy_m BETWEEN 91 AND 100 THEN 10
      ELSE 11
    END AS sort_key
  FROM `bqproj-435911.Final_Project_2025.geolocation`
  WHERE accuracy_m IS NOT NULL
)
SELECT
  accuracy_bucket,
  COUNT(*) AS n_pings
FROM bucketed
GROUP BY accuracy_bucket, sort_key
ORDER BY sort_key;
```

-- 1.1B: Payment method distribution (Yavne Branch)

-- **Purpose:** Identify dominant customer payment behaviors to inform checkout operations and payment infrastructure priorities.

-- **Method:** Aggregate all completed transactions by payment_method and compute each method's share (%) of total transactions.

-- **Result:** Credit_card = 43.83% | Debit_card = 43.33% | Cash = 6.45% | Mobile_pay = 6.38%.

-- **Interpretation:** Over 93% of transactions are cashless, confirming

- - a highly digitized checkout environment at the
- - Yavne branch; cash remains a minority use case.

```
SELECT
    payment_method,
    COUNT(*) AS n_transactions,
    ROUND(
        100 * COUNT(*) / SUM(COUNT(*)) OVER (),
        2
    ) AS pct_transactions
FROM `bqproj-435911.Final_Project_2025.log_sales`
GROUP BY payment_method
ORDER BY n_transactions DESC;
```

-- 1.1C: Average Transaction Value by Hour (Yavne Branch)

-- **Purpose:** Understand how customer basket size (₪) varies by hour to distinguish between peak-volume hours and high-value purchasing periods.

-- **Method:** Aggregate completed transactions by hour of day and compute transaction count, average transaction value, and total revenue.

-- **Result:** • Avg basket is lowest in early morning (08:00-09:00): ~₪300-₪350.

- - • Core shopping hours (10:00-15:00) show stable averages: ~₪470-₪590.
- - • Late hours (19:00-21:00) show sharp spikes (₪750-₪1,000+), driven by very low transaction counts.

-- **Interpretation:** Basket size and transaction volume peak at different times;

- - late-hour averages are volatile and should be interpreted
- - alongside transaction counts, not in isolation.

```
SELECT
    EXTRACT(HOUR FROM timestamp) AS hour_num,
    COUNT(*) AS transactions,
    ROUND(AVG(total), 0) AS avg_transaction_value,
    ROUND(SUM(total), 0) AS total_revenue
FROM `bqproj-435911.Final_Project_2025.log_sales`
GROUP BY hour_num
ORDER BY hour_num;
```

-- 1.1D: Average transaction value vs. transaction volume by hour (Yavne Branch)

-- **Purpose:** Examine the relationship between transaction volume and average basket size to identify whether high-value sales coincide with peak customer activity.

```
-- Method:          Aggregate completed transactions by hour; compute total
transactions and average transaction value (₪) per hour.
-- Result:          Peak transaction volumes (≈3,500–5,800 transactions/hour)
align with moderate average basket values (≈₪470–₪540),
--                - while very high averages (₪750–₪1,020) occur only during low-
volume hours (<1,000 transactions).
-- Interpretation:  High average transaction values during low-volume hours
are driven by small sample sizes rather than broad customer behavior;
--                - revenue stability is primarily generated during high-volume
hours with consistent basket sizes.
```

```
SELECT
  EXTRACT(HOUR FROM timestamp) AS hour_num,
  COUNT(*) AS transactions,
  ROUND(AVG(total), 0) AS avg_transaction_value
FROM `bqproj-435911.Final_Project_2025.log_sales`
GROUP BY hour_num
ORDER BY hour_num;
```

-- 1.1E: Weekly revenue vs. transactions (Yavne Branch)

```
-- Purpose:          Quantify weekly transactions and revenue, and flag weeks
that deviate from normal trading patterns.
-- Method:          Aggregate log_sales by business week ending Friday;
compute transactions, total_revenue, and avg_transaction_value.
-- Result:          Most weeks cluster around ~1.3K–1.4K transactions and
~₪600K–₪630K revenue; low week 2025-11-28 (~511, ~₪253K); early partial week
2025-05-30 (~916, ~₪425K).
-- Interpretation:  Revenue largely scales with transaction volume; treat
holiday/partial weeks as calendar effects when assessing underlying trends.
```

```
WITH weekly AS (
  SELECT
    DATE_TRUNC DATE(timestamp), WEEK(FRIDAY)) AS week_end_friday,
    COUNT(*) AS transactions,
    ROUND(SUM(total), 0) AS total_revenue,
    ROUND(AVG(total), 0) AS avg_transaction_value
  FROM `bqproj-435911.Final_Project_2025.log_sales`
  GROUP BY week_end_friday
)
```

```
SELECT
  week_end_friday,
  transactions,
  total_revenue,
  avg_transaction_value
FROM weekly
ORDER BY week_end_friday;
```

-- 1.1F: Average transaction value by payment method (Yavne Branch)

```
-- Purpose:          Compare basket size across payment methods to understand
whether payment choice correlates with customer spend.
```

```
-- Method:          Aggregate completed transactions by payment_method and
compute transaction count, average transaction value (₪), and total revenue.
-- Result:          • Credit_card: 15,162 transactions | avg ₪520 | total
₪7.89M
--                  - • Debit_card: 14,989 transactions | avg ₪523 | total ₪7.84M
--                  - • Mobile_pay: 2,206 transactions | avg ₪188 | total ₪0.42M
--                  - • Cash: 2,232 transactions | avg ₪184 | total ₪0.41M
-- Interpretation:   Card-based payments are associated with significantly
higher basket sizes than cash or mobile payments, indicating that higher-value
purchases are primarily completed via credit and debit cards.
```

```
SELECT
    payment_method,
    COUNT(*) AS transactions,
    ROUND(AVG(total), 0) AS avg_transaction_value,
    ROUND(SUM(total), 0) AS total_revenue
FROM `bqproj-435911.Final_Project_2025.log_sales`
GROUP BY payment_method
ORDER BY avg_transaction_value DESC;
```

-- Part 2. Metrics

-- 2.1.1: Customer device universe (geolocation)

```
-- Purpose:          Establish the full universe of identifiable customer
devices observed in-store (device_id) to serve as the baseline for customer
segmentation.
-- Method:           Select DISTINCT non-null device_id values from
geolocation, limited to customer-related roles only (repeat_customer /
one_time_customer / not_paying / no_phone).
-- Result:           Returned 1,037 distinct device_ids in your run.
--                  - Observed ID patterns: rep_* (repeat), one_* (one-time), not_*
(non-paying visitors); no_phone does not appear as a device_id here (handled
via log_sales where customer_id IS NULL).
-- Interpretation:   Defines the base output used in the relevant section of
the report.
```

```
SELECT DISTINCT
    device_id
FROM `bqproj-435911.Final_Project_2025.geolocation`
WHERE device_id IS NOT NULL
    AND role IN ('repeat_customer', 'one_time_customer', 'not_paying', 'no_phone')
ORDER BY device_id;
```

-- 2.1.2 – Payers universe (log_sales; customer_id IS NOT NULL)

```
-- Purpose:          Define the set of app-identified customers who completed
≥1 purchase (payer population for behavioural segmentation).
-- Method:           Select DISTINCT customer_id from log_sales where
customer_id is present; alias to device_id for consistent joins to geolocation.
-- Result:           Returns the unique payer device_id list (your run: 761
distinct payers).
```

-- **Interpretation:** Note: Excludes "no_phone" customers by design because their customer_id is NULL.

```
SELECT DISTINCT
  customer_id AS device_id
FROM `bqproj-435911.Final_Project_2025.log_sales`
WHERE customer_id IS NOT NULL
ORDER BY device_id;
```

-- 2.1.3 - No-phone activity (log_sales where customer_id IS NULL)

-- **Purpose:** Quantify purchases made without the app (customer_id IS NULL) to size the "no_phone" segment in transactions and revenue, since these customers cannot be linked to geolocation device_ids.

-- **Method:** Filter log_sales to customer_id IS NULL and aggregate transaction count + revenue totals (subtotal, tax, total) plus time coverage (first/last sale timestamps).

-- **Result:** n_sales_no_phone=3,596 | total_no_phone=₪1,005,656 | subtotal_no_phone=₪852,252 | tax_no_phone=₪153,405 | first_sale_ts=2025-06-01 09:56 | last_sale_ts=2025-11-30 18:17.

-- **Interpretation:** Note: "no_phone" is measurable only as transactions/revenue (not unique customers) because no identifier exists to dedupe individuals.

```
SELECT
  COUNT(DISTINCT sale_id) AS n_sales_no_phone,
  ROUND(SUM(subtotal), 0) AS subtotal_no_phone,
  ROUND(SUM(tax), 0) AS tax_no_phone,
  ROUND(SUM(total), 0) AS total_no_phone,
  MIN(timestamp) AS first_sale_ts,
  MAX(timestamp) AS last_sale_ts
FROM `bqproj-435911.Final_Project_2025.log_sales`
WHERE customer_id IS NULL;
```

-- 2.1.4: In-store presence features per device

-- **Purpose:** Quantify customer visit intensity using geolocation data to support behavioral segmentation (repeat vs. one-time vs. non-paying).

-- **Method:** Count in-store geolocation pings per device_id within SUPERMARKET and CASH_REGISTERS areas, and derive visit frequency metrics: • n_pings_in_store = total in-store pings • n_visit_days = distinct calendar days with in-st...

-- **Result:** Returned 1,036 device_ids with clear separation in visit patterns:

- • rep_* devices show high consistency (typically ~25-27 visit weeks, ~70-100 days, ~2,000+ pings).
- • not_* devices show moderate presence (often ~8-20 weeks with lower ping volumes).
- • one_* devices are mostly low-frequency visitors (often 1 visit week / 1 day).

-- **Interpretation:** Visit frequency metrics provide strong behavioral signals for distinguishing repeat customers from one-time and non-paying visitors and are used directly in the final customer-type classification.


```
WITH customer_instore AS (
  SELECT
    device_id,
    DATE(timestamp) AS visit_date,
    FORMAT_DATE('%G-%V', DATE(timestamp)) AS visit_week
  FROM `bqproj-435911.Final_Project_2025.geolocation`
  WHERE device_id IS NOT NULL
    AND role IN ('repeat_customer', 'one_time_customer', 'not_paying', 'no_phone')
    AND area IN ('SUPERMARKET', 'CASH_REGISTERS')
)
```

```
SELECT
  device_id,
  COUNT(*) AS n_pings_in_store,
  COUNT(DISTINCT visit_date) AS n_visit_days,
  COUNT(DISTINCT visit_week) AS n_visit_weeks
FROM customer_instore
GROUP BY device_id
ORDER BY n_visit_weeks DESC, n_visit_days DESC, n_pings_in_store DESC;
```

-- 2.1.5: Final customer-type classification

```
-- Purpose:      Assign each observable customer to exactly one
behavioural category (repeat_customer / one_time_customer / not_paying /
no_phone) using visit frequency and payment evidence.
-- Method:       Combine in-store visit features (visit weeks) with a
payment flag derived from log_sales; append a no_phone proxy for transactions
with customer_id IS NULL.
-- Result:       Total classified records = 4,632.
--               - Breakdown:
--                 - • repeat_customer : 617
--                 - • one_time_customer : 143
--                 - • not_paying : 276
--                 - • no_phone : 3,596 (transactions, not deduped customers)
-- Interpretation: Most revenue-generating customers are not device-
identifiable
--               - (no_phone), while among app users, repeat customers form the
--               - dominant paying segment. This table serves as the canonical
--               - customer segmentation for downstream analysis.
```

```
WITH device_visits AS (
  SELECT
    device_id,
    COUNT(*) AS n_pings_in_store,
    COUNT(DISTINCT DATE(timestamp)) AS n_visit_days,
    COUNT(DISTINCT FORMAT_DATE('%G-%V', DATE(timestamp))) AS n_visit_weeks
  FROM `bqproj-435911.Final_Project_2025.geolocation`
  WHERE device_id IS NOT NULL
    AND area IN ('SUPERMARKET', 'CASH_REGISTERS')
    AND role IN ('repeat_customer', 'one_time_customer', 'not_paying', 'no_phone')
  GROUP BY device_id
),
```

```

payers AS (
  SELECT DISTINCT
    customer_id AS device_id
  FROM `bqproj-435911.Final_Project_2025.log_sales`
  WHERE customer_id IS NOT NULL
),

device_classified AS (
  SELECT
    v.device_id AS customer_key,
    v.n_pings_in_store,
    v.n_visit_days,
    v.n_visit_weeks,
    IF(p.device_id IS NOT NULL, 1, 0) AS paid_with_app,
    CASE
      WHEN p.device_id IS NULL THEN 'not_paying'
      WHEN v.n_visit_weeks = 1 THEN 'one_time_customer'
      ELSE 'repeat_customer'
    END AS customer_type
  FROM device_visits v
  LEFT JOIN payers p
    ON v.device_id = p.device_id
),

no_phone_proxy AS (
  SELECT
    CONCAT('no_phone_txn_', CAST(sale_id AS STRING)) AS customer_key,
    NULL AS n_pings_in_store,
    NULL AS n_visit_days,
    NULL AS n_visit_weeks,
    0 AS paid_with_app,
    'no_phone' AS customer_type
  FROM `bqproj-435911.Final_Project_2025.log_sales`
  WHERE customer_id IS NULL
)

SELECT
  customer_key,
  customer_type,
  paid_with_app,
  n_visit_weeks,
  n_visit_days,
  n_pings_in_store
FROM device_classified

UNION ALL

SELECT
  customer_key,
  customer_type,
  paid_with_app,
  n_visit_weeks,

```

```

n_visit_days,
n_pings_in_store
FROM no_phone_proxy

ORDER BY
customer_type,
customer_key;

WITH device_visits AS (
SELECT
device_id,
COUNT(*) AS n_pings_in_store,
COUNT(DISTINCT DATE(timestamp)) AS n_visit_days,
COUNT(DISTINCT FORMAT_DATE('%G-%V', DATE(timestamp))) AS n_visit_weeks
FROM `bqproj-435911.Final_Project_2025.geolocation`
WHERE device_id IS NOT NULL
AND area IN ('SUPERMARKET', 'CASH_REGISTERS')
AND role IN ('repeat_customer', 'one_time_customer', 'not_paying', 'no_phone')
GROUP BY device_id
),

payers AS (
SELECT DISTINCT
customer_id AS device_id
FROM `bqproj-435911.Final_Project_2025.log_sales`
WHERE customer_id IS NOT NULL
),

device_classified AS (
SELECT
v.device_id AS customer_key,
v.n_pings_in_store,
v.n_visit_days,
v.n_visit_weeks,
IF(p.device_id IS NOT NULL, 1, 0) AS paid_with_app,
CASE
WHEN p.device_id IS NULL THEN 'not_paying'
WHEN v.n_visit_weeks = 1 THEN 'one_time_customer'
ELSE 'repeat_customer'
END AS customer_type
FROM device_visits v
LEFT JOIN payers p
ON v.device_id = p.device_id
),

no_phone_proxy AS (
SELECT
CONCAT('no_phone_txn_', CAST(sale_id AS STRING)) AS customer_key,
NULL AS n_pings_in_store,
NULL AS n_visit_days,
NULL AS n_visit_weeks,
0 AS paid_with_app,
'no_phone' AS customer_type

```

```
FROM `bqproj-435911.Final_Project_2025.log_sales`
WHERE customer_id IS NULL
)
```

```
SELECT
  customer_key,
  customer_type,
  paid_with_app,
  n_visit_weeks,
  n_visit_days,
  n_pings_in_store
FROM device_classified
```

```
UNION ALL
```

```
SELECT
  customer_key,
  customer_type,
  paid_with_app,
  n_visit_weeks,
  n_visit_days,
  n_pings_in_store
FROM no_phone_proxy
```

```
ORDER BY
  customer_type,
  customer_key;
```

-- 2.1.6: QA summary counts by customer_type (final validation)

```
-- Purpose:      Validate the final customer mapping by reporting
population sizes for each required customer_type and reconciling totals.
-- Method:       Classify device-based customers via geolocation +
log_sales join (repeat/one_time/not_paying), then add "no_phone" as sales rows
with customer_id IS NULL.
-- Result:       repeat_customer=618 | one_time_customer=143 |
not_paying=276 | TOTAL_DEVICE_CUSTOMERS=1,037 | no_phone=3,596 (transactions
proxy; not unique customers).
-- Interpretation: Defines the base output used in the relevant section of
the report.
```

```
WITH device_universe AS (
  SELECT DISTINCT
    device_id
  FROM `bqproj-435911.Final_Project_2025.geolocation`
  WHERE device_id IS NOT NULL
    AND role IN ('repeat_customer', 'one_time_customer', 'not_paying',
'no_phone')
),
```

```
payers AS (
  SELECT DISTINCT
    customer_id AS device_id
```

```

FROM `bqproj-435911.Final_Project_2025.log_sales`
WHERE customer_id IS NOT NULL
),

store_presence AS (
  SELECT
    device_id,
    COUNT(DISTINCT FORMAT_DATE('%G-%V', DATE(timestamp))) AS n_visit_weeks
  FROM `bqproj-435911.Final_Project_2025.geolocation`
  WHERE device_id IS NOT NULL
    AND area IN ('SUPERMARKET', 'CASH_REGISTERS')
    AND role IN ('repeat_customer', 'one_time_customer', 'not_paying',
'no_phone')
  GROUP BY device_id
),

device_classification AS (
  SELECT
    d.device_id AS customer_key,
    CASE
      WHEN p.device_id IS NULL THEN 'not_paying'
      WHEN sp.n_visit_weeks = 1 THEN 'one_time_customer'
      ELSE 'repeat_customer'
    END AS customer_type
  FROM device_universe d
  LEFT JOIN payers p
    ON d.device_id = p.device_id
  LEFT JOIN store_presence sp
    ON d.device_id = sp.device_id
),

device_counts AS (
  SELECT
    customer_type,
    COUNT(DISTINCT customer_key) AS n_customers
  FROM device_classification
  GROUP BY customer_type
),

no_phone_sales AS (
  SELECT
    'no_phone' AS customer_type,
    COUNT(*) AS n_customers
  FROM `bqproj-435911.Final_Project_2025.log_sales`
  WHERE customer_id IS NULL
)

SELECT * FROM device_counts
UNION ALL
SELECT * FROM no_phone_sales
UNION ALL
SELECT
  'TOTAL_DEVICE_CUSTOMERS' AS customer_type,

```

```
SUM(n_customers) AS n_customers
FROM device_counts
ORDER BY
CASE
    WHEN customer_type = 'TOTAL_DEVICE_CUSTOMERS' THEN 3
    WHEN customer_type = 'no_phone' THEN 2
    ELSE 1
END,
n_customers DESC;
```

-- 2.2.1: Employee device universe (geolocation)

-- **Purpose:** Establish the baseline set of identifiable employee devices (one device_id ≈ one employee) for internal segmentation.

-- **Method:** Select DISTINCT non-null device_id from geolocation restricted to known employee roles (manager/cashier/butcher/general/security/delivery).

-- **Result:** Returned 43 distinct employee device_ids (43 rows; 0 nulls; 0 duplicates) – this is the canonical employee universe for Q2 analyses.

-- **Interpretation:** Defines the base output used in the relevant section of the report.

```
SELECT DISTINCT
    device_id
FROM `bqproj-435911.Final_Project_2025.geolocation`
WHERE device_id IS NOT NULL
    AND role IN (
        'manager', 'cashier', 'butcher', 'general_worker',
        'senior_general_worker', 'security_guy', 'delivery_guy'
    )
ORDER BY device_id;
```

-- 2.2.2: Employee role × area exposure (evidence layer)

-- **Purpose:** Build an area-footprint evidence table for each employee device to support role validation and later behavioural role inference.

-- **Method:** Aggregate geolocation pings by (device_id × role × area) across core operational areas, counting the number of observed pings in each area.

-- **Result:** Returned 44 rows covering 43 distinct employee devices.

-- - One employee appears in two operational areas; all others appear in exactly one.

-- - This confirms limited cross-area movement and provides a clean evidence base

-- - for dominant-area assignment in the next step (2.2.3).

-- **Interpretation:** Defines the base output used in the relevant section of the report.

```
SELECT
    device_id,
    role,
    area,
    COUNT(*) AS n_pings_in_area
```

```
FROM `bqproj-435911.Final_Project_2025.geolocation`
WHERE device_id IS NOT NULL
  AND role IN (
    'manager', 'cashier', 'butcher', 'general_worker',
    'senior_general_worker', 'security_guy', 'delivery_guy'
  )
  AND area IN (
    'SUPERMARKET', 'CASH_REGISTERS', 'WAREHOUSE',
    'BUTCHERY', 'PARKING', 'HEAD_OFFICE'
  )
GROUP BY device_id, role, area
ORDER BY device_id, n_pings_in_area DESC;
```

-- 2.2.3: Dominant work area per employee

-- **Purpose:** Assign each employee a single dominant_area based on where most operational-area pings occur (clean anchor for employee mapping and later role/segment inference).

-- **Method:** (1) Build canonical employee device set from geolocation using employee roles. (2) Count pings per (device_id × area) across core work areas.

-- **Result:** 43 employees (43 devices). Dominant areas:
 -- CASH_REGISTERS=15, SUPERMARKET=11, WAREHOUSE=8, BUTCHERY=4, PARKING=4, HEAD_OFFICE=1.
 -- Dominance is extremely strong: 42/43 devices have dominance_share=1.000 (exclusive to one area);
 -- 1/43 device has dominance_share=0.938 (still clearly dominant but with minor exposure to another area).

-- **Interpretation:** Defines the base output used in the relevant section of the report.

```
WITH employee_devices AS (
  SELECT DISTINCT device_id
  FROM `bqproj-435911.Final_Project_2025.geolocation`
  WHERE device_id IS NOT NULL
    AND role IN (
      'manager', 'cashier', 'butcher', 'general_worker',
      'senior_general_worker', 'security_guy', 'delivery_guy'
    )
),
area_pings AS (
  SELECT
    g.device_id,
    g.area,
    COUNT(*) AS n_pings_in_area
  FROM `bqproj-435911.Final_Project_2025.geolocation` g
  JOIN employee_devices e
    ON g.device_id = e.device_id
  WHERE g.area IN (
    'SUPERMARKET', 'CASH_REGISTERS', 'WAREHOUSE',
    'BUTCHERY', 'PARKING', 'HEAD_OFFICE'
  )
  GROUP BY g.device_id, g.area
),
```

```

ranked AS (
    SELECT
        device_id,
        area,
        n_pings_in_area,
        SUM(n_pings_in_area) OVER (PARTITION BY device_id) AS total_pings,
        SAFE_DIVIDE(n_pings_in_area, SUM(n_pings_in_area) OVER (PARTITION BY
device_id)) AS dominance_share,
        ROW_NUMBER() OVER (PARTITION BY device_id ORDER BY n_pings_in_area DESC,
area) AS rn
    FROM area_pings
)
SELECT
    device_id,
    area AS dominant_area,
    n_pings_in_area AS dominant_pings,
    total_pings AS total_pings_all_areas,
    ROUND(dominance_share, 3) AS dominance_share
FROM ranked
WHERE rn = 1
ORDER BY dominant_pings DESC, device_id;

```

-- 2.2.4: Seniority / consistency signals per employee

```

-- Purpose:      Quantify work-consistency signals (weeks/days/intensity)
per employee device to support senior vs non-senior differentiation and
strengthen role inference with behavioural stability evidence.
-- Method:      Filter geolocation to employee roles, then aggregate to
device_id level: - intensity: total pings (n_pings_work) and avg_pings_per_day
- consistency: distinct work days and ISO work weeks - presence window:
first/last se...
-- Result:      Returned 43 rows (one per employee device) spanning 2025-
06-01 to 2025-11-30.
--              - Consistency ranges: n_work_weeks 8-27 (median 21), n_work_days
9-151 (median 37).
--              - Intensity ranges: n_pings_work 100-42,644 (median 6,983) and
avg_pings_per_day 11.1-335.9 (median 156.7).
--              - Practical signal: security devices show the highest ping
intensity (~301-336 avg pings/day),
--              - while delivery devices show the lowest (~11-12 avg pings/day),
suggesting sporadic presence.
-- Interpretation: Defines the base output used in the relevant section of
the report.

```

```

WITH employee_pings AS (
    SELECT
        device_id,
        timestamp
    FROM `bqproj-435911.Final_Project_2025.geolocation`
    WHERE device_id IS NOT NULL
    AND role IN (
        'manager', 'cashier', 'butcher', 'general_worker',
        'senior_general_worker', 'security_guy', 'delivery_guy'
    )
)

```



```
)
SELECT
    device_id,
    COUNT(*) AS n_pings_work,
    COUNT(DISTINCT DATE(timestamp)) AS n_work_days,
    COUNT(DISTINCT FORMAT_DATE('%G-%V', DATE(timestamp))) AS n_work_weeks,
    MIN(timestamp) AS first_seen_ts,
    MAX(timestamp) AS last_seen_ts,
    DATE_DIFF(DATE(MAX(timestamp)), DATE(MIN(timestamp)), DAY) + 1 AS
active_span_days,
    ROUND(SAFE_DIVIDE(COUNT(*), NULLIF(COUNT(DISTINCT DATE(timestamp)), 0)), 1)
AS avg_pings_per_day
FROM employee_pings
GROUP BY device_id
ORDER BY n_work_weeks DESC, n_work_days DESC, n_pings_work DESC;
```

-- 2.2.5: Role inference (dominant area + seniority rule)

-- **Purpose:** Infer a functional employee role per device_id using observed behaviour: dominant operational area + consistency (weeks) as a seniority signal.

-- **Method:** (1) Identify employee devices from geolocation using employee-role filter (baseline universe). (2) Compute dominant_area per device based on ping concentration across operational areas.

-- **Result:** Returned 43 inferred employees (one per device_id) with 0 'unknown' assignments.

-- - Distribution: cashier=15, general_worker=10, delivery_guy=8, butcher=4,

-- - security_guy=4, manager=1, senior_general_worker=1.

-- **Interpretation:** Defines the base output used in the relevant section of the report.

```
WITH employee_devices AS (
    SELECT DISTINCT device_id
    FROM `bqproj-435911.Final_Project_2025.geolocation`
    WHERE device_id IS NOT NULL
    AND role IN (
        'manager', 'cashier', 'butcher', 'general_worker',
        'senior_general_worker', 'security_guy', 'delivery_guy'
    )
),
area_pings AS (
    SELECT
        g.device_id,
        g.area,
        COUNT(*) AS n_pings_in_area
    FROM `bqproj-435911.Final_Project_2025.geolocation` g
    JOIN employee_devices e ON g.device_id = e.device_id
    WHERE g.area IN (
        'SUPERMARKET', 'CASH_REGISTERS', 'WAREHOUSE',
        'BUTCHERY', 'PARKING', 'HEAD_OFFICE'
    )
    GROUP BY g.device_id, g.area
),
```

```

dominant_area AS (
    SELECT
        device_id,
        area AS dominant_area,
        SAFE_DIVIDE(n_pings_in_area, SUM(n_pings_in_area) OVER (PARTITION BY
device_id)) AS dominance_share
    FROM area_pings
    QUALIFY ROW_NUMBER() OVER (PARTITION BY device_id ORDER BY n_pings_in_area
DESC, area) = 1
),
seniority AS (
    SELECT
        g.device_id,
        COUNT(DISTINCT FORMAT_DATE('%G-%V', DATE(g.timestamp))) AS n_work_weeks,
        COUNT(DISTINCT DATE(g.timestamp)) AS n_work_days
    FROM `bqproj-435911.Final_Project_2025.geolocation` g
    JOIN employee_devices e ON g.device_id = e.device_id
    GROUP BY g.device_id
)
SELECT
    e.device_id,
    CASE
        WHEN d.dominant_area = 'HEAD_OFFICE' THEN 'manager'
        WHEN d.dominant_area = 'CASH_REGISTERS' THEN 'cashier'
        WHEN d.dominant_area = 'BUTCHERY' THEN 'butcher'
        WHEN d.dominant_area = 'PARKING' THEN 'security_guy'
        WHEN d.dominant_area = 'WAREHOUSE' THEN 'delivery_guy'
        WHEN d.dominant_area = 'SUPERMARKET' AND s.n_work_weeks >= 26 THEN
'senior_general_worker'
        WHEN d.dominant_area = 'SUPERMARKET' THEN 'general_worker'
        ELSE 'unknown'
    END AS inferred_role,
    d.dominant_area,
    ROUND(d.dominance_share, 3) AS dominance_share,
    s.n_work_weeks,
    s.n_work_days
FROM employee_devices e
LEFT JOIN dominant_area d USING (device_id)
LEFT JOIN seniority s USING (device_id)
ORDER BY inferred_role, e.device_id;

```

-- 2.2.6: Final employee role mapping (one row per employee)

```

-- Purpose:      Produce the canonical internal segmentation table (one
row per employee device) for downstream staffing/ops analysis.
-- Method:       Reuse the behavioural inference logic from 2.2.5 inside a
CTE, then output only the final mapping plus explainability fields (dominant
area, dominance share, and consistency metrics: work weeks/days, active span).
-- Result:       Returned 43 mapped employee devices (reconciles exactly
to the 43-device universe in 2.2.1) with 0 'unknown' roles.
--               - Role distribution: cashier=15, general_worker=10,
delivery_guy=8, butcher=4, security_guy=4, manager=1, senior_general_worker=1.
--               - Dominance is extremely strong: 42/43 devices have
dominance_share=1.000; 1/43 has dominance_share=0.938.

```

-- Consistency ranges: n_work_weeks 8-27 (median 21), n_work_days 9-151 (median 37), active_span_days 141-183 (median 175).
 -- **Interpretation:** Defines the base output used in the relevant section of the report.

```
WITH inferred AS (
  -- (same logic as 2.2.5; kept as a CTE so the final output is clean)
  WITH employee_devices AS (
    SELECT DISTINCT device_id
    FROM `bqproj-435911.Final_Project_2025.geolocation`
    WHERE device_id IS NOT NULL
    AND role IN (
      'manager', 'cashier', 'butcher', 'general_worker',
      'senior_general_worker', 'security_guy', 'delivery_guy'
    )
  ),
  area_pings AS (
    SELECT
      g.device_id,
      g.area,
      COUNT(*) AS n_pings_in_area
    FROM `bqproj-435911.Final_Project_2025.geolocation` g
    JOIN employee_devices e ON g.device_id = e.device_id
    WHERE g.area IN (
      'SUPERMARKET', 'CASH_REGISTERS', 'WAREHOUSE',
      'BUTCHERY', 'PARKING', 'HEAD_OFFICE'
    )
    GROUP BY g.device_id, g.area
  ),
  dominant_area AS (
    SELECT
      device_id,
      area AS dominant_area,
      SAFE_DIVIDE(n_pings_in_area, SUM(n_pings_in_area) OVER (PARTITION BY
device_id)) AS dominance_share
    FROM area_pings
    QUALIFY ROW_NUMBER() OVER (PARTITION BY device_id ORDER BY n_pings_in_area
DESC, area) = 1
  ),
  seniority AS (
    SELECT
      g.device_id,
      COUNT(DISTINCT FORMAT_DATE('%G-%V', DATE(g.timestamp))) AS n_work_weeks,
      COUNT(DISTINCT DATE(g.timestamp)) AS n_work_days,
      DATE_DIFF(MAX(DATE(g.timestamp)), MIN(DATE(g.timestamp)), DAY) + 1 AS
active_span_days
    FROM `bqproj-435911.Final_Project_2025.geolocation` g
    JOIN employee_devices e ON g.device_id = e.device_id
    GROUP BY g.device_id
  )
  SELECT
    e.device_id,
    CASE
      WHEN d.dominant_area = 'HEAD_OFFICE' THEN 'manager'
```

```

        WHEN d.dominant_area = 'CASH_REGISTERS' THEN 'cashier'
        WHEN d.dominant_area = 'BUTCHERY' THEN 'butcher'
        WHEN d.dominant_area = 'PARKING' THEN 'security_guy'
        WHEN d.dominant_area = 'WAREHOUSE' THEN 'delivery_guy'
        WHEN d.dominant_area = 'SUPERMARKET' AND s.n_work_weeks >= 26 THEN
'senior_general_worker'
        WHEN d.dominant_area = 'SUPERMARKET' THEN 'general_worker'
        ELSE 'unknown'
    END AS inferred_role,
    d.dominant_area,
    ROUND(d.dominance_share, 3) AS dominance_share,
    s.n_work_weeks,
    s.n_work_days,
    s.active_span_days
FROM employee_devices e
LEFT JOIN dominant_area d USING (device_id)
LEFT JOIN seniority s USING (device_id)
)
SELECT
    device_id AS employee_device_id,
    inferred_role,
    dominant_area,
    dominance_share,
    n_work_weeks,
    n_work_days,
    active_span_days
FROM inferred
ORDER BY inferred_role, employee_device_id;

```

-- 2.2.7 (BONUS): Infer employee/customer segment WITHOUT geolocation.role

-- **Purpose:** Classify each device_id using only observed behaviour (area footprint + regularity) and payment evidence (log_sales), without using geolocation.role.

-- **Method:** (1) Build store-level pings for all non-null device_ids across operational areas. (2) Aggregate device-level activity signals: total pings, active days/weeks, activity span, and avg pings/day.

-- **Result:** Classified 1,080 total devices.

-- - Customer segments: repeat_customer=617, one_time_customer=135, not_paying=276.

-- - Employee-like segments inferred (no role used): cashier=15, general_worker=10,

-- - delivery_guy=8, butcher=4, manager=1, senior_general_worker=1, security_guy=13

-- - (total employee-like=52).

-- **Interpretation:** Note: Employee-like total (52) exceeds the role-filtered employee universe (43 in 2.2.1),

```

WITH store_pings AS (
    SELECT
        device_id,
        area,
        timestamp
    FROM `bqproj-435911.Final_Project_2025.geolocation`

```

```

WHERE device_id IS NOT NULL
AND area IN (
    'SUPERMARKET', 'CASH_REGISTERS', 'WAREHOUSE', 'BUTCHERY',
    'ENTRANCE', 'BACK_OFFICE', 'HEAD_OFFICE', 'PARKING'
)
),

device_time AS (
SELECT
    device_id,
    COUNT(*) AS n_total_pings,
    COUNT(DISTINCT DATE(timestamp)) AS n_active_days,
    COUNT(DISTINCT FORMAT_DATE('%G-%V', DATE(timestamp))) AS n_active_weeks,
    MIN(timestamp) AS first_seen_ts,
    MAX(timestamp) AS last_seen_ts,
    DATE_DIFF(DATE(MAX(timestamp)), DATE(MIN(timestamp)), DAY) + 1 AS
active_span_days,
    SAFE_DIVIDE(COUNT(*), NULLIF(COUNT(DISTINCT DATE(timestamp)), 0)) AS
avg_pings_per_day
FROM store_pings
GROUP BY device_id
),

area_counts AS (
SELECT
    device_id,
    area,
    COUNT(*) AS n_pings_in_area
FROM store_pings
GROUP BY device_id, area
),

dominant_area AS (
SELECT
    device_id,
    area AS dominant_area,
    SAFE_DIVIDE(n_pings_in_area,
        SUM(n_pings_in_area) OVER (PARTITION BY device_id)
    ) AS dominance_share
FROM area_counts
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY device_id
    ORDER BY n_pings_in_area DESC, area
) = 1
),

payers AS (
SELECT DISTINCT
    customer_id AS device_id
FROM `bqproj-435911.Final_Project_2025.log_sales`
WHERE customer_id IS NOT NULL
),

```

```

base AS (
  SELECT
    t.device_id,
    t.n_total_pings,
    t.n_active_days,
    t.n_active_weeks,
    t.first_seen_ts,
    t.last_seen_ts,
    t.active_span_days,
    t.avg_pings_per_day,
    d.dominant_area,
    ROUND(d.dominance_share, 3) AS dominance_share,
    IF(p.device_id IS NOT NULL, 1, 0) AS paid_with_app
  FROM device_time t
  LEFT JOIN dominant_area d USING (device_id)
  LEFT JOIN payers p USING (device_id)
),

labeled AS (
  SELECT
    *,
    CASE
      -- employee-like (no payment + strong regularity)
      WHEN dominant_area = 'HEAD_OFFICE' THEN 'manager'
      WHEN dominant_area = 'CASH_REGISTERS' THEN 'cashier'
      WHEN dominant_area = 'BUTCHERY' THEN 'butcher'
      WHEN dominant_area = 'WAREHOUSE' THEN 'delivery_guy'
      WHEN dominant_area = 'PARKING' THEN 'security_guy'
      WHEN dominant_area = 'SUPERMARKET'
        AND n_active_weeks >= 26 THEN 'senior_general_worker'
      WHEN dominant_area = 'SUPERMARKET'
        AND n_active_weeks < 26 THEN 'general_worker'

      -- customer types
      WHEN paid_with_app = 1 AND n_active_days = 1 THEN 'one_time_customer'
      WHEN paid_with_app = 1 THEN 'repeat_customer'
      ELSE 'not_paying'
    END AS inferred_segment
  FROM base
)

SELECT
  device_id,
  inferred_segment,
  dominant_area,
  dominance_share,
  n_active_weeks,
  n_active_days,
  active_span_days,
  avg_pings_per_day,
  paid_with_app
FROM labeled
ORDER BY inferred_segment, device_id;

```

-- 2.3.1: Accuracy outliers (accuracy_m > 30m)

-- **Purpose:** Quantify low-accuracy broadcasts and identify where they concentrate (by role and area).

-- **Method:** From geolocation pings with non-null device_id and accuracy_m, count outliers (accuracy_m > 30), compute outlier share, and summarize accuracy distribution (avg/median/p90/p99) using grouping sets to return overall, by r...

-- **Result:** Overall 1,308,624 pings; 415,580 outliers → outlier_share = 31.76%.

- - Concentration: PARKING has the highest outlier_share (54.79%, p99≈114m); CASH_REGISTERS is lowest (14.20%).
- - Role hotspot: security_guy is consistently worst (outlier_share 88.65%, avg≈82m, p99≈149m),
- - consistent with outdoor/edge-of-building reception.

-- **Interpretation:** Defines the base output used in the relevant section of the report.

```
WITH base AS (
  SELECT
    device_id,
    role,
    area,
    accuracy_m
  FROM `bqproj-435911.Final_Project_2025.geolocation`
  WHERE device_id IS NOT NULL
    AND accuracy_m IS NOT NULL
)
SELECT
  IFNULL(role, 'ALL_ROLES') AS role,
  IFNULL(area, 'ALL_AREAS') AS area,
  COUNT(*) AS n_pings,
  COUNTIF(accuracy_m > 30) AS n_outlier_pings,
  ROUND(SAFE_DIVIDE(COUNTIF(accuracy_m > 30), COUNT(*)), 4) AS outlier_share,
  ROUND(AVG(accuracy_m), 2) AS avg_accuracy_m,
  APPROX_QUANTILES(accuracy_m, 100)[OFFSET(50)] AS median_accuracy_m,
  APPROX_QUANTILES(accuracy_m, 100)[OFFSET(90)] AS p90_accuracy_m,
  APPROX_QUANTILES(accuracy_m, 100)[OFFSET(99)] AS p99_accuracy_m
FROM base
GROUP BY GROUPING SETS
  ((role, area), (role), (area), ())
ORDER BY
  outlier_share DESC,
  n_outlier_pings DESC,
  n_pings DESC;
```

-- 2.3.2: App performance (PARKING reception vs 40% benchmark)

-- **Purpose:** Verify whether the app meets the design expectation of 1 broadcast per minute once a device is in PARKING, and whether observed reception is at least 40% of possible per-minute broadcasts.

```
-- Method:          (1) Filter geolocation to PARKING pings and bucket
timestamps to minutes. (2) Sessionize per device (new session if minute gap >
10 minutes).
-- Result:          n_parking_sessions=26,790;
n_devices_with_parking_activity=995.
--          - capture_rate_weighted=0.5250 (>=0.40, meets benchmark overall);
--          - capture_rate_avg_session=0.7301; pct_sessions_below_40=0.0604
(≈6.04%).
-- Interpretation:   Defines the base output used in the relevant section of
the report.
```

```
WITH parking AS (
  SELECT
    device_id,
    TIMESTAMP_TRUNC(timestamp, MINUTE) AS minute_ts
  FROM `bqproj-435911.Final_Project_2025.geolocation`
  WHERE device_id IS NOT NULL
    AND timestamp IS NOT NULL
    AND area = 'PARKING'
),

with_prev AS (
  SELECT
    device_id,
    minute_ts,
    LAG(minute_ts) OVER (PARTITION BY device_id ORDER BY minute_ts) AS
prev_minute_ts
  FROM parking
),

flags AS (
  SELECT
    device_id,
    minute_ts,
    CASE
      WHEN prev_minute_ts IS NULL THEN 1
      WHEN TIMESTAMP_DIFF(minute_ts, prev_minute_ts, MINUTE) > 10 THEN 1
      ELSE 0
    END AS is_new_session
  FROM with_prev
),

sessioned AS (
  SELECT
    device_id,
    minute_ts,
    SUM(is_new_session) OVER (PARTITION BY device_id ORDER BY minute_ts) AS
session_id
  FROM flags
),

sessions AS (
  SELECT
```



```

device_id,
session_id,
COUNT(DISTINCT minute_ts) AS observed_minutes,
1 + TIMESTAMP_DIFF(MAX(minute_ts), MIN(minute_ts), MINUTE) AS
expected_minutes,
SAFE_DIVIDE(
COUNT(DISTINCT minute_ts),
1 + TIMESTAMP_DIFF(MAX(minute_ts), MIN(minute_ts), MINUTE)
) AS capture_rate
FROM sessioned
GROUP BY device_id, session_id
),

final AS (
SELECT
COUNT(*) AS n_parking_sessions,
COUNT(DISTINCT device_id) AS n_devices_with_parking_activity,
SAFE_DIVIDE(SUM(observed_minutes), SUM(expected_minutes)) AS
capture_rate_weighted,
AVG(capture_rate) AS capture_rate_avg_session,
AVG(IF(capture_rate < 0.40, 1, 0)) AS pct_sessions_below_40
FROM sessions
)

SELECT
n_parking_sessions,
n_devices_with_parking_activity,
ROUND(capture_rate_weighted, 4) AS capture_rate_weighted,
ROUND(capture_rate_avg_session, 4) AS capture_rate_avg_session,
ROUND(pct_sessions_below_40, 4) AS pct_sessions_below_40,
(capture_rate_weighted < 0.40) AS weighted_is_below_40
FROM final;

```

-- 2.3.3 (BONUS): Additional outliers beyond accuracy_m

```

-- Purpose: Identify other device-level anomalies (beyond accuracy_m)
that could bias analysis, focusing on extreme ping volume and extreme ping
intensity.
-- Method: Aggregate geolocation to device_id level (n_pings,
n_active_days, active_span_days, avg_pings_per_day). Flag outliers using an IQR
upper-fence rule (upper_fence = Q3 + 1.5*IQR) for: (1) n_pings (volume
outliers) (2) avg_...
-- Result: Returned 38 outlier devices.
-- - Thresholds in this run: upper_fence_pings=5,811.5 and
upper_fence_intensity=42.2.
-- - Of the 38 devices: 30 exceeded both volume and intensity; 8
exceeded intensity only.
-- - The flagged set is dominated by employee devices
(cashiers/general workers/butchers/security + manager + senior),
-- - which is expected because employees have near-daily presence and
high ping intensity.
-- - Additionally, 3 one_time_customer devices were flagged on
intensity only (1 active day inflates avg/day).

```

-- - Implication: these devices are not representative of typical customer behaviour; if not excluded from
 -- - customer-level analyses, they will materially skew averages and visit/intensity metrics.
 -- **Interpretation:** Defines the base output used in the relevant section of the report.

```
WITH device_metrics AS (
  SELECT
    device_id,
    COUNT(*) AS n_pings,
    COUNT(DISTINCT DATE(timestamp)) AS n_active_days,
    DATE_DIFF(MAX(DATE(timestamp)), MIN(DATE(timestamp)), DAY) + 1 AS
active_span_days,
    SAFE_DIVIDE(COUNT(*), NULLIF(COUNT(DISTINCT DATE(timestamp)), 0)) AS
avg_pings_per_day
  FROM `bqproj-435911.Final_Project_2025.geolocation`
  WHERE device_id IS NOT NULL
    AND timestamp IS NOT NULL
  GROUP BY device_id
),
bounds AS (
  SELECT
    APPROX_QUANTILES(n_pings, 4)[OFFSET(1)] AS q1_pings,
    APPROX_QUANTILES(n_pings, 4)[OFFSET(3)] AS q3_pings,
    APPROX_QUANTILES(avg_pings_per_day, 4)[OFFSET(1)] AS q1_intensity,
    APPROX_QUANTILES(avg_pings_per_day, 4)[OFFSET(3)] AS q3_intensity
  FROM device_metrics
),
scored AS (
  SELECT
    d.*,
    (b.q3_pings + 1.5 * (b.q3_pings - b.q1_pings)) AS upper_fence_pings,
    (b.q3_intensity + 1.5 * (b.q3_intensity - b.q1_intensity)) AS
upper_fence_intensity,
    (d.n_pings > (b.q3_pings + 1.5 * (b.q3_pings - b.q1_pings))) AS
is_outlier_ping_volume,
    (d.avg_pings_per_day > (b.q3_intensity + 1.5 * (b.q3_intensity -
b.q1_intensity))) AS is_outlier_intensity
  FROM device_metrics d
  CROSS JOIN bounds b
)
SELECT
  device_id,
  n_pings,
  n_active_days,
  active_span_days,
  ROUND(avg_pings_per_day, 1) AS avg_pings_per_day,
  is_outlier_ping_volume,
  is_outlier_intensity,
  ROUND(upper_fence_pings, 1) AS upper_fence_pings,
  ROUND(upper_fence_intensity, 1) AS upper_fence_intensity
FROM scored
WHERE is_outlier_ping_volume
```

```

    OR is_outlier_intensity
ORDER BY n_pings DESC, avg_pings_per_day DESC;
WITH device_metrics AS (
    SELECT
        device_id,
        COUNT(*) AS n_pings,
        COUNT(DISTINCT DATE(timestamp)) AS n_active_days,
        DATE_DIFF(MAX(DATE(timestamp)), MIN(DATE(timestamp)), DAY) + 1 AS
active_span_days,
        SAFE_DIVIDE(COUNT(*), NULLIF(COUNT(DISTINCT DATE(timestamp)), 0)) AS
avg_pings_per_day
    FROM `bqproj-435911.Final_Project_2025.geolocation`
    WHERE device_id IS NOT NULL
        AND timestamp IS NOT NULL
    GROUP BY device_id
),

bounds AS (
    SELECT
        -- IQR bounds for n_pings
        APPROX_QUANTILES(n_pings, 4)[OFFSET(1)] AS q1_pings,
        APPROX_QUANTILES(n_pings, 4)[OFFSET(3)] AS q3_pings,

        -- IQR bounds for avg_pings_per_day
        APPROX_QUANTILES(avg_pings_per_day, 4)[OFFSET(1)] AS q1_intensity,
        APPROX_QUANTILES(avg_pings_per_day, 4)[OFFSET(3)] AS q3_intensity
    FROM device_metrics
),

scored AS (
    SELECT
        d.*,

        (b.q3_pings - b.q1_pings) AS iqr_pings,
        (b.q3_pings + 1.5 * (b.q3_pings - b.q1_pings)) AS upper_fence_pings,

        (b.q3_intensity - b.q1_intensity) AS iqr_intensity,
        (b.q3_intensity + 1.5 * (b.q3_intensity - b.q1_intensity)) AS
upper_fence_intensity,

        -- Outlier flags
        (d.n_pings > (b.q3_pings + 1.5 * (b.q3_pings - b.q1_pings))) AS
is_outlier_ping_volume,
        (d.avg_pings_per_day > (b.q3_intensity + 1.5 * (b.q3_intensity -
b.q1_intensity))) AS is_outlier_intensity

    FROM device_metrics d
    CROSS JOIN bounds b
)

SELECT
    device_id,
    n_pings,

```

```

n_active_days,
active_span_days,
ROUND(avg_pings_per_day, 1) AS avg_pings_per_day,

-- which rules triggered
is_outlier_ping_volume,
is_outlier_intensity,

-- thresholds for transparency
ROUND(upper_fence_pings, 1) AS upper_fence_pings,
ROUND(upper_fence_intensity, 1) AS upper_fence_intensity

FROM scored
WHERE is_outlier_ping_volume
      OR is_outlier_intensity
ORDER BY n_pings DESC, avg_pings_per_day DESC;

-- 2.4.1: Identify supplier (delivery) devices (role = delivery_guy)
-- Purpose:      Produce the list of supplier devices that arrive for
scheduled deliveries (Mon/Thu ~06:00).
-- Method:      Filter geolocation to role='delivery_guy', restrict to
delivery days (Mon/Thu) and the 06:00 window (05:30-06:30), and focus on
delivery-relevant areas (WAREHOUSE/ENTRANCE/PARKING).
-- Result:      Returned 8 supplier devices (del_001-del_008).
delivery_window_pings range: 95-171 per device;
--      - n_delivery_dates range: 9-16 per device. Time coverage:
first_seen_ts=2025-06-05 (~06:00 UTC),
--      - last_seen_ts=2025-11-24 (~06:29 UTC).
-- Interpretation: Defines the base output used in the relevant section of
the report.

WITH delivery_window AS (
  SELECT
    device_id,
    DATE(timestamp) AS dt,
    timestamp
  FROM `bqproj-435911.Final_Project_2025.geolocation`
  WHERE device_id IS NOT NULL
        AND role = 'delivery_guy'
        AND area IN ('WAREHOUSE', 'ENTRANCE', 'PARKING')
        AND EXTRACT(DAYOFWEEK FROM DATE(timestamp)) IN (2, 5) -- Mon, Thu
        AND TIME(timestamp) BETWEEN TIME '05:30:00' AND TIME '06:30:00'
)
SELECT
  device_id,
  COUNT(*) AS delivery_window_pings,
  COUNT(DISTINCT dt) AS n_delivery_dates,
  MIN(timestamp) AS first_seen_ts,
  MAX(timestamp) AS last_seen_ts
FROM delivery_window
GROUP BY device_id
ORDER BY n_delivery_dates DESC, delivery_window_pings DESC, device_id;

```

-- 2.4.2 (BONUS): Infer supplier (delivery) devices WITHOUT using geolocation.role

-- **Purpose:** Identify supplier devices behaviorally (ignore role) using the scheduled delivery signature: Monday/Thursday around ~06:00 (05:30-06:30) in delivery-relevant areas.

-- **Method:** Filter geolocation to (Mon/Thu) × (05:30-06:30) × (WAREHOUSE/ENTRANCE/PARKING), then aggregate to device_id and require repeat appearances across multiple delivery dates.

-- **Result:** Returned 8 inferred supplier devices (del_001-del_008), matching the delivery cohort found via role.

-- - Consistency: n_delivery_dates = 9-16 per device;
delivery_window_pings = 95-171.

-- - Time coverage: first_seen_ts = 2025-06-05 05:56 UTC;
last_seen_ts = 2025-11-24 06:29 UTC.

-- **Interpretation:** Defines the base output used in the relevant section of the report.

```
WITH candidate_pings AS (
  SELECT
    device_id,
    DATE(timestamp) AS dt,
    timestamp
  FROM `bqproj-435911.Final_Project_2025.geolocation`
  WHERE device_id IS NOT NULL
    AND area IN ('WAREHOUSE', 'ENTRANCE', 'PARKING')
    AND EXTRACT(DAYOFWEEK FROM DATE(timestamp)) IN (2, 5) -- Mon, Thu
    AND TIME(timestamp) BETWEEN TIME '05:30:00' AND TIME '06:30:00'
)

SELECT
  device_id,
  COUNT(*) AS delivery_window_pings,
  COUNT(DISTINCT dt) AS n_delivery_dates,
  MIN(timestamp) AS first_seen_ts,
  MAX(timestamp) AS last_seen_ts
FROM candidate_pings
GROUP BY device_id
WHERE COUNT(DISTINCT dt) >= 5
ORDER BY n_delivery_dates DESC, delivery_window_pings DESC, device_id;
```

-- 2.4.3: Scheduled delivery dates with NO supplier arrival

-- **Purpose:** Identify Monday/Thursday delivery dates (~06:00) where no supplier device arrived, i.e., no delivery_guy devices were observed in the delivery window/areas.

-- **Method:** (1) Build the scheduled delivery calendar from the dataset date range (all Mon/Thu dates). (2) For each scheduled date, count distinct supplier devices (role='delivery_guy') observed in delivery-relevant areas (WAREHOUSE/ENTRANCE/PARKING) within ±30 minutes of 06:00.

-- **Result:** Missed scheduled deliveries (no supplier arrival): 2025-06-02, 2025-06-26, 2025-08-07,
 -- - 2025-10-02, 2025-11-27 (all with n_supplier_devices=0).
 -- **Interpretation:** Defines the base output used in the relevant section of the report.

```
WITH date_range AS (
  SELECT
    MIN(DATE(timestamp)) AS min_d,
    MAX(DATE(timestamp)) AS max_d
  FROM `bqproj-435911.Final_Project_2025.geolocation`
),
scheduled AS (
  SELECT
    d AS scheduled_date,
    TIMESTAMP(DATETIME(d, TIME '06:00:00')) AS scheduled_ts
  FROM date_range,
  UNNEST(GENERATE_DATE_ARRAY(min_d, max_d)) AS d
  WHERE EXTRACT(DAYOFWEEK FROM d) IN (2, 5) -- Mon, Thu
),
arrivals AS (
  SELECT
    s.scheduled_date,
    COUNT(DISTINCT g.device_id) AS n_supplier_devices,
    COUNT(*) AS n_supplier_pings
  FROM scheduled s
  LEFT JOIN `bqproj-435911.Final_Project_2025.geolocation` g
    ON DATE(g.timestamp) = s.scheduled_date
    AND g.device_id IS NOT NULL
    AND g.role = 'delivery_guy'
    AND g.area IN ('WAREHOUSE', 'ENTRANCE', 'PARKING')
    AND g.timestamp BETWEEN TIMESTAMP_SUB(s.scheduled_ts, INTERVAL 30 MINUTE)
    AND TIMESTAMP_ADD(s.scheduled_ts, INTERVAL 30 MINUTE)
  GROUP BY s.scheduled_date
)
SELECT
  scheduled_date,
  n_supplier_devices,
  n_supplier_pings
FROM arrivals
WHERE n_supplier_devices = 0
ORDER BY scheduled_date;
```

-- 2.5.1: Top 5 customers by visit frequency

-- **Purpose:** Identify the five customers who visit the supermarket most frequently across the period.
 -- **Method:** Use geolocation as the presence signal, restrict to customer roles (exclude employees), and count distinct visit days per device_id (visit_days). Rank by visit_days (tie-break by total pings).
 -- **Result:** Top 5 by visit_days:
 -- - cus_003 = 146, cus_011 = 139, cus_021 = 133, cus_045 = 129, cus_078 = 125.

-- **Interpretation:** Defines the base output used in the relevant section of the report.

```
WITH customer_instore AS (
  SELECT
    device_id,
    DATE(timestamp) AS visit_date
  FROM `bqproj-435911.Final_Project_2025.geolocation`
  WHERE device_id IS NOT NULL
    AND role IN ('repeat_customer', 'one_time_customer', 'not_paying', 'no_phone')
    AND area IN ('SUPERMARKET', 'CASH_REGISTERS')
)
SELECT
  device_id,
  COUNT(DISTINCT visit_date) AS visit_days,
  COUNT(*) AS total_instore_pings
FROM customer_instore
GROUP BY device_id
ORDER BY visit_days DESC, total_instore_pings DESC, device_id
LIMIT 5;
```

-- 2.5.2: Top 5 customers by cumulative spend

-- **Purpose:** Identify the five customers with the highest total spending across the full period (app-attributed customers only).

-- **Method:** Use log_sales as the transaction source, exclude NULL customer_id, aggregate to customer_id: n_transactions, total_spend = SUM(total), avg_basket = AVG(total), first/last purchase timestamps.

-- **Result:** Top 5 by cumulative spend (total_spend):

-- - rep_207 = 51,527.53 (98 tx), rep_182 = 51,237.57 (100 tx),

-- - rep_076 = 50,578.71 (93 tx),

-- - rep_171 = 49,391.87 (92 tx), rep_217 = 49,181.26 (90 tx).

-- **Interpretation:** Defines the base output used in the relevant section of the report.

```
SELECT
  customer_id AS device_id,
  COUNT(*) AS n_transactions,
  ROUND(SUM(total), 2) AS total_spend,
  ROUND(AVG(total), 2) AS avg_basket,
  MIN(timestamp) AS first_purchase_ts,
  MAX(timestamp) AS last_purchase_ts
FROM `bqproj-435911.Final_Project_2025.log_sales`
WHERE customer_id IS NOT NULL
GROUP BY customer_id
ORDER BY total_spend DESC, n_transactions DESC, device_id
LIMIT 5;
```

```
SELECT
  customer_id AS device_id,
  COUNT(*) AS n_transactions,
  ROUND(SUM(total), 2) AS total_spend,
  ROUND(AVG(total), 2) AS avg_basket,
```

```

    MIN(timestamp) AS first_purchase_ts,
    MAX(timestamp) AS last_purchase_ts
FROM `bqproj-435911.Final_Project_2025.log_sales`
WHERE customer_id IS NOT NULL
GROUP BY customer_id
ORDER BY total_spend DESC, n_transactions DESC, device_id
LIMIT 5;

```

-- 2.6.1: Customer presence by hour × weekday (simple aggregate)

-- **Purpose:** Build the core dataset for weekday × hour customer presence (for heatmap + operating-hours inference).

-- **Method:** Exclude employees, count DISTINCT customer device_ids per weekday and hour.

-- **Result:** One row per (weekday × hour) where at least 1 customer device was observed.

-- **Interpretation:** Defines the base output used in the relevant section of the report.

```

SELECT
CASE EXTRACT(DAYOFWEEK FROM DATE(timestamp))
  WHEN 1 THEN 'Sunday'
  WHEN 2 THEN 'Monday'
  WHEN 3 THEN 'Tuesday'
  WHEN 4 THEN 'Wednesday'
  WHEN 5 THEN 'Thursday'
  WHEN 6 THEN 'Friday'
  WHEN 7 THEN 'Saturday'
END AS weekday_name,
EXTRACT(DAYOFWEEK FROM DATE(timestamp)) AS weekday_num,
EXTRACT(HOUR FROM timestamp) AS hour,
COUNT(DISTINCT device_id) AS n_customers
FROM `bqproj-435911.Final_Project_2025.geolocation`
WHERE device_id IS NOT NULL
  AND role NOT IN (
    'manager', 'cashier', 'butcher', 'general_worker',
    'senior_general_worker', 'security_guy', 'delivery_guy'
  )
GROUP BY weekday_name, weekday_num, hour
ORDER BY weekday_num, hour;

```

-- 2.6.2: Customer presence by hour × weekday (filtered for “real operating hours”)

-- **Purpose:** Remove low-volume noise hours and approximate the supermarket’s real operating hours based on customer presence.

-- **Method:** Use geolocation, exclude employees, count DISTINCT customer device_ids per weekday and hour, and keep only hours with meaningful traffic (HAVING n_customers >= 15).

-- **Result:** Output is the set of weekday-hours that meet the traffic threshold (use to infer opening/closing windows).

-- **Interpretation:** Defines the base output used in the relevant section of the report.


```

SELECT
  CASE EXTRACT(DAYOFWEEK FROM DATE(timestamp))
    WHEN 1 THEN 'Sunday'
    WHEN 2 THEN 'Monday'
    WHEN 3 THEN 'Tuesday'
    WHEN 4 THEN 'Wednesday'
    WHEN 5 THEN 'Thursday'
    WHEN 6 THEN 'Friday'
    WHEN 7 THEN 'Saturday'
  END AS weekday_name,
  EXTRACT(DAYOFWEEK FROM DATE(timestamp)) AS weekday_num,
  EXTRACT(HOUR FROM timestamp) AS hour,
  COUNT(DISTINCT device_id) AS n_customers
FROM `bqproj-435911.Final_Project_2025.geolocation`
WHERE device_id IS NOT NULL
  -- exclude employees (customer presence only)
  AND role NOT IN (
    'manager', 'cashier', 'butcher', 'general_worker',
    'senior_general_worker', 'security_guy', 'delivery_guy'
  )
GROUP BY weekday_name, weekday_num, hour
HAVING COUNT(DISTINCT device_id) >= 15
ORDER BY weekday_num, hour;

```

-- Part 3. Business Recommendations (Customer Exp. & Operations)

```

-- 3A.2.1: Paying-customer load by weekday × hour (log_sales)
-- Purpose:      Quantify checkout demand by identifying when paying
customers and transactions peak across the week.
-- Method:      Use log_sales as the ground-truth for checkout workload;
group transactions by weekday × hour and compute: (1) n_paying_customers =
COUNT(DISTINCT customer_id) (2) n_transactions = COUNT(*) Exclude NULL
customer_id becau...
-- Result:      Returned 59 weekday-hour rows (hours with sales activity
only; no Saturday activity appears in output).
--              - Peak workload windows:
--              - • Friday 14:00 = 2,487 transactions (377 paying customers) -
highest transaction hour.
--              - • Friday 13:00 = 399 paying customers (1,869 transactions) -
highest paying-customer hour.
--              - Secondary hotspots concentrate on Thursday:
--              - • Thursday 19:00 = 1,341 transactions (371 paying customers)
--              - • Thursday 11:00 = 1,309 transactions (381 paying customers)
--              - Quiet tail: Thursday 21:00 = 5 transactions (5 paying
customers).
--              - Takeaway: Demand is highly time-concentrated (especially Thu +
Fri), making it suitable for targeted staffing
--              - and self-checkout deployment in peak windows.

```

-- **Interpretation:** Defines the base output used in the relevant section of the report.

```
SELECT
  FORMAT_DATE('%A', DATE(timestamp)) AS weekday_name,
  EXTRACT(DAYOFWEEK FROM DATE(timestamp)) AS weekday_num, -- Sun=1 ... Sat=7
  EXTRACT(HOUR FROM timestamp) AS hour,
  COUNT(DISTINCT customer_id) AS n_paying_customers,
  COUNT(*) AS n_transactions
FROM `bqproj-435911.Final_Project_2025.log_sales`
WHERE customer_id IS NOT NULL
GROUP BY 1,2,3
ORDER BY weekday_num, hour;
```

-- 3A.3.1: Cashiers needed per weekday × hour (from log_sales)

-- **Purpose:** Recommend the optimal number of staffed cashiers each hour of the week using actual checkout workload.

-- **Method:** Use log_sales as ground-truth; for each weekday × hour, compute the distribution of transactions per hour across dates (AVG and P90).

-- **Result:** Peak staffing is concentrated on Thu + Fri.

- - • Fri 14:00: P90 = 106 tx/hour → 5 cashiers (highest hour).
- - • Fri 10:00: P90 = 76 tx/hour → 4 cashiers; Fri 13:00: P90 = 79 → 4 cashiers.
- - • Thu peaks: 11:00 (P90 59 → 3), 13:00 (P90 59 → 3), 17:00 (P90 62 → 3), 19:00 (P90 61 → 3).
- - All other hours are typically 1–2 cashiers at P90, with a clear mid-afternoon lull (e.g., Thu 15:00 P90=2 → 1).
- - Takeaway: A small set of predictable peak windows drives cashier needs; staffing can be tightly scheduled by hour.

-- **Interpretation:** Defines the base output used in the relevant section of the report.

```
WITH sales_by_day_hour AS (
  SELECT
    DATE(timestamp) AS dt,
    FORMAT_DATE('%A', DATE(timestamp)) AS weekday_name,
    EXTRACT(DAYOFWEEK FROM DATE(timestamp)) AS weekday_num, -- Sun=1 ...
Sat=7
    EXTRACT(HOUR FROM timestamp) AS hour,
    COUNT(*) AS tx_per_hour
  FROM `bqproj-435911.Final_Project_2025.log_sales`
  WHERE customer_id IS NOT NULL
  GROUP BY dt, weekday_name, weekday_num, hour
),
summary AS (
  SELECT
    weekday_name,
    weekday_num,
    hour,
    AVG(tx_per_hour) AS avg_tx_per_hour,
    APPROX_QUANTILES(tx_per_hour, 100)[OFFSET(90)] AS p90_tx_per_hour
  FROM sales_by_day_hour
```

```

    GROUP BY weekday_name, weekday_num, hour
)
SELECT
    weekday_name,
    weekday_num,
    hour,
    ROUND(avg_tx_per_hour, 1) AS avg_tx_per_hour,
    p90_tx_per_hour,
    25 AS tx_per_cashier_per_hour,
    CEIL(avg_tx_per_hour / 25) AS recommended_cashiers_avg,
    CEIL(p90_tx_per_hour / 25) AS recommended_cashiers_p90
FROM summary
ORDER BY weekday_num, hour;

```

-- 3A.3.2: Checkout load by weekday × hour (log_sales)

-- **Purpose:** Quantify when checkout demand is highest/lowest by day and hour using actual transactions.

-- **Method:** Group log_sales by weekday × hour and compute: -
 n_paying_customers = COUNT(DISTINCT customer_id) - n_transactions = COUNT(*)
 Exclude NULL customer_id (cannot be tied to a customer/device).

-- **Result:** 59 weekday-hour rows returned (hours with sales activity only).

-- - Peaks: Fri 14:00 = 2,487 tx; Fri 13:00 = 399 paying customers (1,869 tx).

-- - Thu peaks at 11:00–14:00 and ~19:00 (e.g., Thu 19:00 = 1,341 tx).

-- - Quiet tail: Thu 21:00 = 5 tx (5 paying customers).

-- **Interpretation:** Defines the base output used in the relevant section of the report.

```

SELECT
    FORMAT_DATE('%A', DATE(timestamp)) AS weekday_name,
    EXTRACT(DAYOFWEEK FROM DATE(timestamp)) AS weekday_num,    -- Sun=1 ... Sat=7
    EXTRACT(HOUR FROM timestamp) AS hour,
    COUNT(DISTINCT customer_id) AS n_paying_customers,
    COUNT(*) AS n_transactions
FROM `bqproj-435911.Final_Project_2025.log_sales`
WHERE customer_id IS NOT NULL
GROUP BY 1,2,3
ORDER BY weekday_num, hour;

```

-- 3A.3.3: Peak-hour flag + self-checkout recommendation (decision layer)

-- **Purpose:** Identify the specific weekday-hours where self-checkout should be prioritised to absorb peak demand and save manpower.

-- **Method:** (1) Build hourly checkout load (transactions/hour). (2) For each weekday, compute a PEAK threshold (Q75 of hourly transactions).

-- **Result:** Self-checkout hours flagged in this run:

-- - • Thursday: 11:00–14:00 and 17:00–20:00

-- - • Friday: 09:00–14:00

-- - • Sunday: 10:00 and 12:00

-- - Non-peak example: Thu 21:00 = 5 tx (no self-checkout needed).

-- **Interpretation:** Defines the base output used in the relevant section of the report.

```
DECLARE min_tx_for_self_checkout INT64 DEFAULT 550;

WITH load AS (
  SELECT
    FORMAT_DATE('%A', DATE(timestamp)) AS weekday_name,
    EXTRACT(DAYOFWEEK FROM DATE(timestamp)) AS weekday_num, -- Sun=1 ... Sat=7
    EXTRACT(HOUR FROM timestamp) AS hour,
    COUNT(DISTINCT customer_id) AS n_paying_customers,
    COUNT(*) AS n_transactions
  FROM `bqproj-435911.Final_Project_2025.log_sales`
  WHERE customer_id IS NOT NULL
  GROUP BY 1,2,3
),
weekday_thresholds AS (
  SELECT
    weekday_num,
    APPROX_QUANTILES(n_transactions, 100)[OFFSET(75)] AS q75_tx
  FROM load
  GROUP BY weekday_num
)
SELECT
  l.weekday_name,
  l.weekday_num,
  l.hour,
  l.n_paying_customers,
  l.n_transactions,
  t.q75_tx,
  CASE WHEN l.n_transactions >= t.q75_tx THEN 'PEAK' ELSE 'NON-PEAK' END AS
load_band,
  (l.n_transactions >= t.q75_tx AND l.n_transactions >=
min_tx_for_self_checkout) AS recommend_self_checkout
FROM load l
JOIN weekday_thresholds t
  USING (weekday_num)
ORDER BY l.weekday_num, l.hour;
```

-- 3A.4.1: Dates with NO supermarket activity (full closure)

-- **Purpose:** Identify calendar dates where the supermarket shows zero operational activity.

-- **Method:** Build a full date calendar from the combined min/max dates across geolocation + log_sales. Left-join daily counts from each table:
 (1) n_geo_pings = COUNT(*) per day from geolocation (2) n_sales_rows = COUNT(*) per day f...

-- **Result:** Full-closure dates observed:

- - 2025-04-13, 2025-04-14, 2025-04-20, 2025-05-26, 2025-06-02,
- - 2025-09-23, 2025-09-24, 2025-10-02, 2025-10-03, 2025-10-12,
- 2025-10-13.
- - Takeaway: These are true "no activity" days (not just missing geolocation or missing sales) and should be

-- - excluded from demand baselines, peak-hour analysis, and staffing optimisation.
 -- **Interpretation:** Defines the base output used in the relevant section of the report.

```
WITH date_bounds AS (
  SELECT
    LEAST(
      (SELECT MIN(DATE(timestamp)) FROM `bqproj-435911.Final_Project_2025.geolocation`),
      (SELECT MIN(DATE(timestamp)) FROM `bqproj-435911.Final_Project_2025.log_sales`)
    ) AS min_d,
    GREATEST(
      (SELECT MAX(DATE(timestamp)) FROM `bqproj-435911.Final_Project_2025.geolocation`),
      (SELECT MAX(DATE(timestamp)) FROM `bqproj-435911.Final_Project_2025.log_sales`)
    ) AS max_d
),

calendar AS (
  SELECT d AS dt
  FROM date_bounds, UNNEST(GENERATE_DATE_ARRAY(min_d, max_d)) AS d
),

geo_daily AS (
  SELECT
    DATE(timestamp) AS dt,
    COUNT(*) AS n_geo_pings
  FROM `bqproj-435911.Final_Project_2025.geolocation`
  GROUP BY dt
),

sales_daily AS (
  SELECT
    DATE(timestamp) AS dt,
    COUNT(*) AS n_sales_rows
  FROM `bqproj-435911.Final_Project_2025.log_sales`
  GROUP BY dt
)

SELECT
  c.dt AS date,
  IFNULL(g.n_geo_pings, 0) AS n_geo_pings,
  IFNULL(s.n_sales_rows, 0) AS n_sales_rows
FROM calendar c
LEFT JOIN geo_daily g USING (dt)
LEFT JOIN sales_daily s USING (dt)
WHERE IFNULL(g.n_geo_pings, 0) = 0
  AND IFNULL(s.n_sales_rows, 0) = 0
ORDER BY date;
```

-- 3A.4.2: Unusually high customer-activity dates (daily spikes)

-- **Purpose:** Flag calendar dates where customer activity was exceptionally high, to identify event-driven demand surges (e.g., holiday eves / promotions) that should be treated separately in staffing and planning.

-- **Method:** From geolocation, compute DAILY distinct customer devices (COUNT DISTINCT device_id), excluding employee roles. Compute the 95th-percentile threshold (p95) across days, then return dates where daily customers $\geq$ p95.

-- **Result:** p95_threshold = 408 distinct customer devices/day.

- - 9 "spike" dates (dt → n_customer_devices):
- - • 2025-09-04 → 420 (highest)
- - • 2025-07-03 → 417
- - • 2025-06-05 → 416
- - • 2025-11-13 → 412
- - • 2025-06-20 → 410
- - • 2025-09-11 → 410
- - • 2025-07-17 → 408
- - • 2025-08-29 → 408
- - • 2025-11-27 → 408

-- **Interpretation:** These are the strongest demand-surge days in the dataset (by distinct customer presence).

- - They likely reflect "special days" (holiday-adjacent / promotion / payday effects) rather than normal demand,
- - so they should be explicitly flagged in operational planning (extra staffing, checkout capacity, inventory readiness).

```
WITH daily_customers AS (
  SELECT
    DATE(timestamp) AS dt,
    COUNT(DISTINCT device_id) AS n_customer_devices
  FROM `bqproj-435911.Final_Project_2025.geolocation`
  WHERE timestamp IS NOT NULL
    AND device_id IS NOT NULL
    -- customers only (explicit, safer than NOT IN)
    AND role IN ('repeat_customer', 'one_time_customer', 'not_paying', 'no_phone')
    -- focus on true in-store presence (avoid parking/edge drift)
    AND area IN ('SUPERMARKET', 'CASH_REGISTERS')
  GROUP BY dt
),
```

```
threshold AS (
  SELECT
    APPROX_QUANTILES(n_customer_devices, 100)[OFFSET(95)] AS p95_threshold
  FROM daily_customers
)
```

```
SELECT
  d.dt,
  d.n_customer_devices,
  t.p95_threshold
FROM daily_customers d
CROSS JOIN threshold t
```

```
WHERE d.n_customer_devices >= t.p95_threshold
ORDER BY d.n_customer_devices DESC, d.dt;
```

-- Part 4. Regression Models: Visits & Spend

-- 4.1: Weekly unique customer devices (Regression 1 dataset)

-- **Purpose:** Build the regression input table where Y = weekly unique customer devices, X = week_index (time).

-- **Method:** Filter to customer roles (as defined in Part 2 Q2), aggregate DISTINCT device_id by week_start, then create a sequential week_index for regression. Output: week_start, week_index, n_unique_devices

-- **Result:** Returns the requested output table for analysis (see query output).

-- **Interpretation:** Defines the base output used in the relevant section of the report.

```
WITH base_customers AS (
  SELECT
    DATE(timestamp) AS dt,
    device_id
  FROM `bqproj-435911.Final_Project_2025.geolocation`
  WHERE device_id IS NOT NULL
  -- Customer universe (match Part 2 Q2 definition)
  AND role IN ('repeat_customer', 'one_time_customer', 'not_paying', 'no_phone')
  -- Optional (recommended): keep true in-store signals (reduce boundary
  drift)
  AND area IN ('SUPERMARKET', 'CASH_REGISTERS')
),

weekly AS (
  SELECT
    DATE_TRUNC(dt, WEEK(MONDAY)) AS week_start,
    COUNT(DISTINCT device_id) AS n_unique_devices
  FROM base_customers
  GROUP BY week_start
),

final AS (
  SELECT
    week_start,
    DENSE_RANK() OVER (ORDER BY week_start) AS week_index,
    n_unique_devices
  FROM weekly
)

SELECT *
FROM final
ORDER BY week_start;
```

-- 4.2: Basket total vs dwell time (dataset for linear regression)

-- **Purpose:** Build the analysis dataset for Regression #2, testing whether time spent in-store (minutes) is associated with purchase amount (total).

-- **Method:** Use log_sales (the only table with dwell_minutes). Keep app-attributed customers only: - customer_id IS NOT NULL (identifiable app customer) - dwell_minutes IS NOT NULL (only exists for app users) - total IS NOT NULL (pu...

-- **Result:** 30,993 transaction rows returned (Regression 2.csv), each with:

-- - customer_id, sale_id, timestamp, dwell_minutes, purchase_total (= total), minutes_per_shekel.

-- **Interpretation:** Note: This output is the correct input to the Python OLS model where:

```
SELECT
  customer_id,
  sale_id,
  timestamp,
  dwell_minutes,
  total AS purchase_total,
  SAFE_DIVIDE(dwell_minutes, total) AS minutes_per_shekel
FROM `bqproj-435911.Final_Project_2025.log_sales`
WHERE customer_id IS NOT NULL
  AND dwell_minutes IS NOT NULL
  AND total IS NOT NULL;
```

-- Part 5 – Looker Studio Dashboard (Charts & Insights)

-- 5.1: Heatmap source: customer activity by weekday x hour (customers only)

-- **Purpose:** Create a heatmap-ready dataset of unique customer device activity by weekday x hour for Looker Studio.

-- **Method:** Filter geolocation to customer roles (exclude employees), restrict to in-store areas (SUPERMARKET, CASH_REGISTERS), group by date + weekday + hour, and COUNT(DISTINCT device_id); keep dt for dashboard date filtering.

-- **Result:** 1,440 rows covering 151 dates (2025-06-01 to 2025-11-30), hours 08-21, weekdays Sunday-Friday (no Saturday rows); n_unique_customer_devices ranges 1-190 (avg ≈ 47.3).

-- **Interpretation:** Use this as the source for the weekdayxhour heatmap; in Looker, aggregate across dt (e.g., AVG for “typical” activity, SUM for total volume) depending on the story you want.

```
SELECT
  DATE(timestamp) AS dt, -- used for dashboard date filters
  EXTRACT(DAYOFWEEK FROM DATE(timestamp)) AS weekday_num, -- Sun=1 ... Sat=7
  FORMAT_DATE('%A', DATE(timestamp)) AS weekday_name,
  EXTRACT(HOUR FROM timestamp) AS hour,
  COUNT(DISTINCT device_id) AS n_unique_customer_devices
```



```
FROM `bqproj-435911.Final_Project_2025.geolocation`
WHERE device_id IS NOT NULL
  -- customers only (avoid employees)
  AND role IN ('repeat_customer', 'one_time_customer', 'not_paying', 'no_phone')
  -- optional: focus on real in-store presence (recommended for cleaner signal)
  AND area IN ('SUPERMARKET', 'CASH_REGISTERS')
GROUP BY dt, weekday_num, weekday_name, hour;
```

-- 5.2: Daily staffing by shift and role (distinct employee devices)

-- **Purpose:** Build a Looker Studio staffing dataset showing daily headcount by shift and employee role (distinct employee devices).

-- **Method:** Filter geolocation to employee roles, derive shift buckets from hour (Morning 06–13, Afternoon 14–17, Evening 18–23), keep hours 06–23 only, then COUNT(DISTINCT device_id) grouped by dt × shift × role.

-- **Result:** 2,447 rows covering 151 dates (2025-06-01 to 2025-11-30), 3 shifts, 7 roles; n_staff per dt×shift×role ranges 1–6 (avg ≈ 1.73). Aggregated across roles, total staff per dt×shift ranges 6–13 (avg ≈ 9.93).

-- **Interpretation:** Use for shift-level staffing visuals (stacked by role or filtered by role); distinct-device counting approximates headcount and supports day/shift comparisons over time.

```
WITH base AS (
  SELECT
    DATE(timestamp) AS dt,
    EXTRACT(HOUR FROM timestamp) AS hr,
    role,
    device_id
  FROM `bqproj-435911.Final_Project_2025.geolocation`
  WHERE role IN (
    'cashier',
    'general_worker',
    'senior_general_worker',
    'manager',
    'butcher',
    'security_guy',
    'delivery_guy'
  )
),
shifted AS (
  SELECT
    dt,
    CASE
      WHEN hr BETWEEN 6 AND 13 THEN 'Morning (06-13)'
      WHEN hr BETWEEN 14 AND 17 THEN 'Afternoon (14-17)'
      WHEN hr BETWEEN 18 AND 23 THEN 'Evening (18-23)'
      ELSE 'Other'
    END AS shift,
    role,
    device_id
  FROM base
  WHERE hr BETWEEN 6 AND 23
```

```
)
SELECT
    dt,
    shift,
    role,
    COUNT(DISTINCT device_id) AS n_staff
FROM shifted
GROUP BY 1,2,3
ORDER BY dt, shift, role;
```

-- 5.3: Weekly unique customer devices (repeat + one_time customers) for trend chart

-- **Purpose:** Produce a weekly trend series of unique customer devices (repeat + one_time) for a Looker Studio line chart.

-- **Method:** Truncate timestamps to week_start (WEEK(MONDAY)), compute year + ISO-style week_num, and COUNT(DISTINCT device_id) per week.

-- **Result:** 27 weekly rows from 2025-05-26 through 2025-11-24; n_unique_customer_devices ranges 171-394 (avg $\approx$ 377.8), with continuous weekly coverage (no missing weeks).

-- **Interpretation:** Use week_start on the X-axis and n_unique_customer_devices as the metric to show weekly customer-device reach over time (repeat + one_time combined).

```
SELECT
    DATE_TRUNC(DATE(timestamp), WEEK(MONDAY)) AS week_start,
    EXTRACT(YEAR FROM DATE(timestamp)) AS year,
    EXTRACT(WEEK(MONDAY) FROM DATE(timestamp)) AS week_num,
    COUNT(DISTINCT device_id) AS n_unique_customer_devices
FROM `bqproj-435911.Final_Project_2025.geolocation`
WHERE role IN ('repeat_customer', 'one_time_customer')
GROUP BY 1, 2, 3
ORDER BY week_start;
```

-- 5.4: Raw customer events (so Looker can COUNT DISTINCT device_id within the selected date range)

-- **Purpose:** Provide raw customer geolocation events so Looker Studio can compute COUNT DISTINCT(device_id) dynamically within any selected date range.

-- **Method:** Filter geolocation to customer roles only (repeat_customer, one_time_customer), map role $\rightarrow$ friendly customer_type label, and output dt + customer_type + device_id at event level (no aggregation).

-- **Interpretation:** Use as the base source for unique-customer KPIs/trends by customer_type; in Looker use COUNT_DISTINCT(device_id) (optionally by dt) to avoid double-counting repeat pings.

```
SELECT
    DATE(timestamp) AS dt,
    CASE
```

```
    WHEN role = 'repeat_customer' THEN 'Returning (repeat)'
    WHEN role = 'one_time_customer' THEN 'Occasional (one-time)'
END AS customer_type,
device_id
FROM `bqproj-435911.Final_Project_2025.geolocation`
WHERE role IN ('repeat_customer', 'one_time_customer');
```

-- APPENDIX 1 - Supporting Queries

-- Appendix 1.1 - Row count snapshot (all raw tables)

-- **Purpose:** Establish baseline table sizes for QA context and downstream sanity checks.

-- **Method:** Count rows in each raw table used in the project.

-- **Result:** Confirms scale of each dataset input (geolocation, log_sales, supermarkets, Lamas).

-- **Interpretation:** Provides a quick snapshot to validate ingestion completeness.

```
SELECT 'geolocation' AS table_name, COUNT(*) AS row_count
FROM `bqproj-435911.Final_Project_2025.geolocation`
```

```
UNION ALL
SELECT 'log_sales', COUNT(*)
FROM `bqproj-435911.Final_Project_2025.log_sales`
```

```
UNION ALL
SELECT 'supermarkets_il', COUNT(*)
FROM `bqproj-435911.Final_Project_2025.supermarkets_il`
```

```
UNION ALL
SELECT 'lamas_israel_2025', COUNT(*)
FROM `bqproj-435911.Final_Project_2025.Lamas_israel_2025`;
```

-- Appendix 1.2 - Time coverage snapshot (min/max timestamps)

-- **Purpose:** Confirm the observed analysis window and detect unexpected extra/missing time coverage.

-- **Method:** Compute MIN/MAX timestamp for geolocation and log_sales.

-- **Result:** Returns each table's time span based on recorded timestamps.

-- **Interpretation:** Ensures report dates and dashboard filters align with true coverage.

```
SELECT
    MIN(timestamp) AS min_ts,
    MAX(timestamp) AS max_ts
FROM `bqproj-435911.Final_Project_2025.geolocation`;
```

```
SELECT
    MIN(timestamp) AS min_ts,
    MAX(timestamp) AS max_ts
FROM `bqproj-435911.Final_Project_2025.log_sales`;
```

-- Appendix 1.3 - Geolocation spatial bounds (lat/lon)

-- **Purpose:** Validate that geolocation reflects a single physical site by checking tight lat/lon bounds plus overall ping + device scale.
 -- **Method:** Full-table aggregation: COUNT(*) pings, COUNT(DISTINCT device_id) devices, and MIN/MAX lat/lon after excluding NULL coordinates.
 -- **Result:** n_pings=1,308,624 | n_devices=1,080 | lat_range=31.877595-31.879135 | lon_range=34.738607-34.740357
 -- **Interpretation:** Tight lat/lon envelope supports a single-location dataset (Yavne); area/role variation likely reflects within-branch zones.

```
SELECT
  COUNT(*) AS n_pings,
  COUNT(DISTINCT device_id) AS n_devices,
  MIN(lat) AS min_lat,
  MAX(lat) AS max_lat,
  MIN(lon) AS min_lon,
  MAX(lon) AS max_lon
FROM `bqproj-435911.Final_Project_2025.geolocation`
WHERE lat IS NOT NULL
      AND lon IS NOT NULL;
```

-- Appendix 1.4 – Schema snapshot (field names & data types)

-- **Purpose:** Document table schemas to support interpretation and prevent type-related errors.
 -- **Method:** Pull column metadata (names, types) for project tables.
 -- **Result:** Outputs the schema structure used across analysis queries.
 -- **Interpretation:** Supports join validation, casting assumptions, and field availability checks.

```
SELECT
  table_name,
  column_name,
  data_type
FROM `bqproj-435911.Final_Project_2025.INFORMATION_SCHEMA.COLUMNS`
WHERE table_name IN ('geolocation', 'log_sales', 'supermarkets_il',
  'Lamas_israel_2025')
ORDER BY table_name, ordinal_position;
```

-- Appendix 1.5 – Null profile for key fields (geolocation + log_sales)

-- **Purpose:** Quantify missingness in critical fields that drive segmentation, timing, and regression.
 -- **Method:** Count NULLs for key columns in geolocation and log_sales.
 -- **Result:** Returns NULL counts per field for quick QA.
 -- **Interpretation:** Highlights fields requiring filters, SAFE handling, or explicit limitations.

```
SELECT
```

```
COUNT(*) AS total_rows,
COUNTIF(device_id IS NULL) AS null_device_id,
COUNTIF(lat IS NULL OR lon IS NULL) AS null_coordinates,
COUNTIF(accuracy_m IS NULL) AS null_accuracy,
COUNTIF(role IS NULL) AS null_role
FROM `bqproj-435911.Final_Project_2025.geolocation`;
```

```
SELECT
COUNT(*) AS total_rows,
COUNTIF(sale_id IS NULL) AS null_sale_id,
COUNTIF(customer_id IS NULL) AS null_customer_id,
COUNTIF(dwelling_minutes IS NULL) AS null_dwelling_minutes,
COUNTIF(total IS NULL) AS null_total
FROM `bqproj-435911.Final_Project_2025.log_sales`;
```

-- Appendix 1.6 - Location accuracy bounds (0-100m window)

-- **Purpose:** Confirm observed min/max accuracy_m within the 0-100m window used for charting consistency.

-- **Method:** Compute MIN/MAX accuracy_m after filtering to non-null values within 0-100m.

-- **Result:** accuracy_m range (0-100m window) = 1.0 → 100.0

-- **Interpretation:** Anchors the bounded accuracy window used for QA and outlier logic.

```
SELECT
MIN(accuracy_m) AS min_accuracy_m,
MAX(accuracy_m) AS max_accuracy_m
FROM `bqproj-435911.Final_Project_2025.geolocation`
WHERE accuracy_m IS NOT NULL
AND accuracy_m BETWEEN 0 AND 100;
```

-- Appendix 1.7 - Location accuracy percentiles (0-100m window)

-- **Purpose:** Quantify typical accuracy and tail behaviour using percentile checkpoints to justify an operational outlier threshold.

-- **Method:** Use APPROX_QUANTILES on accuracy_m (0-100m) and extract P50/P90/P95/P99.

-- **Result:** P50=17.6m | P90=38.9m | P95=42.3m | P99=57.8m

-- **Interpretation:** Supports the selected accuracy threshold and explains tail impact.

```
SELECT
APPROX_QUANTILES(accuracy_m, 100)[OFFSET(50)] AS p50_accuracy_m,
APPROX_QUANTILES(accuracy_m, 100)[OFFSET(90)] AS p90_accuracy_m,
APPROX_QUANTILES(accuracy_m, 100)[OFFSET(95)] AS p95_accuracy_m,
APPROX_QUANTILES(accuracy_m, 100)[OFFSET(99)] AS p99_accuracy_m
FROM `bqproj-435911.Final_Project_2025.geolocation`
```

```
WHERE accuracy_m IS NOT NULL
      AND accuracy_m BETWEEN 0 AND 100;
```

-- Appendix 1.8 - Outlier share by 30m threshold (0-100m window)

-- **Purpose:** Measure the share of high-confidence pings ($\leq 30m$) versus lower-quality pings ($> 30m$) to quantify outlier impact.

-- **Method:** Compute % $\leq 30m$ and % $> 30m$ in the 0-100m window using COUNTIF.

-- **Result:** pct_ $\leq 30m$ =74.2% | pct_ $> 30m$ =25.8%

-- **Interpretation:** Quantifies the expected noise share if $\leq 30m$ is used as a reliability threshold.

```
SELECT
  ROUND(100 * COUNTIF(accuracy_m <= 30) / COUNT(*), 1) AS pct_le_30m,
  ROUND(100 * COUNTIF(accuracy_m > 30) / COUNT(*), 1) AS pct_gt_30m
FROM `bqproj-435911.Final_Project_2025.geolocation`
WHERE accuracy_m IS NOT NULL
      AND accuracy_m BETWEEN 0 AND 100;
```

-- Appendix 1.9 - Identifier sanity checks (distinct counts)

-- **Purpose:** Confirm core identifiers behave as expected before aggregation and KPI reporting.

-- **Method:** Compute distinct counts for sale_id (log_sales) and device_id (geolocation).

-- **Result:** Returns total rows vs distinct IDs for quick integrity verification.

-- **Interpretation:** Supports confidence that distinct-based KPIs are not inflated by ID reuse.

```
SELECT
  COUNT(*) AS total_rows,
  COUNT(DISTINCT sale_id) AS distinct_sale_ids
FROM `bqproj-435911.Final_Project_2025.log_sales`;
```

```
SELECT
  COUNT(DISTINCT device_id) AS distinct_devices
FROM `bqproj-435911.Final_Project_2025.geolocation`;
```

-- Appendix 1.10 - Duplicate sale_id check (log_sales)

-- **Purpose:** Confirm sales records are uniquely keyed to prevent revenue/dwell inflation.

-- **Method:** Group by sale_id and filter to counts > 1 .

-- **Result:** Returns duplicate sale_ids if present (otherwise zero rows).

-- **Interpretation:** Validates that log_sales can be safely aggregated without deduping.

```
SELECT sale_id, COUNT(*) AS cnt
```

```
FROM `bqproj-435911.Final_Project_2025.log_sales`
GROUP BY sale_id
HAVING COUNT(*) > 1
ORDER BY cnt DESC
LIMIT 20;
```

-- Appendix 1.11 – Duplicate device_id + timestamp check (geolocation)

-- **Purpose:** Detect exact duplicate pings that can bias visit intensity and hourly heatmaps.

-- **Method:** Group by device_id + timestamp and return any combinations with count > 1.

-- **Result:** Returns duplicate ping keys if present (otherwise zero rows).

-- **Interpretation:** Confirms event granularity is stable; duplicates would require deduping rules.

```
SELECT
    device_id,
    timestamp,
    COUNT(*) AS duplicate_count
FROM `bqproj-435911.Final_Project_2025.geolocation`
GROUP BY device_id, timestamp
HAVING COUNT(*) > 1
ORDER BY duplicate_count DESC
LIMIT 20;
```

-- Appendix 1.12 – Duplicate supermarket_id check (supermarkets_il)

-- **Purpose:** Validate dimension-table key uniqueness to prevent join multiplication.

-- **Method:** Group by supermarket_id and filter to counts > 1.

-- **Result:** Returns duplicated supermarket_id keys if present (otherwise zero rows).

-- **Interpretation:** Confirms safe joins between event tables and supermarket metadata.

```
SELECT
    id AS supermarket_id,
    COUNT(*) AS duplicate_count
FROM `bqproj-435911.Final_Project_2025.supermarkets_il`
GROUP BY id
HAVING COUNT(*) > 1
ORDER BY duplicate_count DESC;
```

-- Appendix 1.13 – Duplicate city_id check (Lamas)

-- Purpose: Validate reference-table key uniqueness for city-level enrichments and normalization.
 -- Method: Group by city_id and filter to counts > 1.
 -- Result: Returns duplicated city_id keys if present (otherwise zero rows).
 -- Interpretation: Prevents join duplication and ensures consistent demographic baselines.

```
SELECT
  city_id,
  COUNT(*) AS duplicate_count
FROM `bqproj-435911.Final_Project_2025.Lamas_israel_2025`
GROUP BY city_id
HAVING COUNT(*) > 1
ORDER BY duplicate_count DESC;
```

-- Appendix 1.14 – Clean-table parity check (geolocation_base_clean vs raw)
 -- **Purpose:** Validate that the cleaned base table preserves expected row volume versus the raw source.
 -- **Method:** Build geolocation_base_clean and compare row counts against raw geolocation.
 -- **Result:** Shows row_count for raw_geolocation vs geolocation_base_clean.
 -- **Interpretation:** Confirms whether cleaning logic drops rows and by how much.

```
WITH geolocation_base_clean AS (
  SELECT
    device_id,
    lat,
    lon,
    timestamp,
    accuracy_m,
    role,
    area,
    DATE(timestamp) AS event_date,
    EXTRACT(DAYOFWEEK FROM timestamp) AS weekday_num,
    EXTRACT(HOUR FROM timestamp) AS hour_num,
    EXTRACT(WEEK FROM DATE(timestamp)) AS week_num
  FROM `bqproj-435911.Final_Project_2025.geolocation`
)
```

```
SELECT
  'raw_geolocation' AS source,
  COUNT(*) AS row_count
FROM `bqproj-435911.Final_Project_2025.geolocation`
```

```
UNION ALL
```

```
SELECT
```

```
'geolocation_base_clean' AS source,  
COUNT(*) AS row_count  
FROM geolocation_base_clean;
```